

HPPS Assignment A5

Björn Petterson (zts900), Manar Haybout(chs380), Thomas Andersen (kgq325)

April 15, 2026

Introduction

This assignment was completed in much less time than previous assignments due to the tight schedule. That means that we generally made use of LLMs for our first draft code. During the development we ran into different issues with the recursion in `nbody-bh` which cost us a lot of time debugging. One of the unfortunate consequences of this was that benchmarks were run on the Barnes-Hut version of our code did not actually produce warnings correctly. We have since been able to fix that issue and believe that we have ended up with working code that will compile without warnings and you can successfully run `benchmark.sh` on it.

Below you will see that benchmarking for `nbody.c` and `nbody-bh.c` was run on different hardware. Our analysis is done in relation to those benchmarks. To get comparable data across benchmarks we have included the output from the script in the handout. You will find those results on the last page of this report in figure .

Specific questions

1. The following questions pertain to `nbody.c`.

- a) *Measure and show the weak and strong scaling of `nbody` as you vary the number of threads. Explain how you chose the datasets.*

We chose 4 different datasets with varying number of particles and ran 100 iterations for each to ensure warnings were generated even with only relatively few particles. We ran each dataset on 1 to 10 cores. What is clear from figure 4 is that the more particles are used in the simulation the better performance scales with additional threads. There is also the clear indication of diminishing

returns, even for the simulation that scaled the best. The datasets were primarily chosen to test the difference between relatively small and large workloads, as we expected to see better scaling for large workloads due to the overhead involved in spinning up new threads.

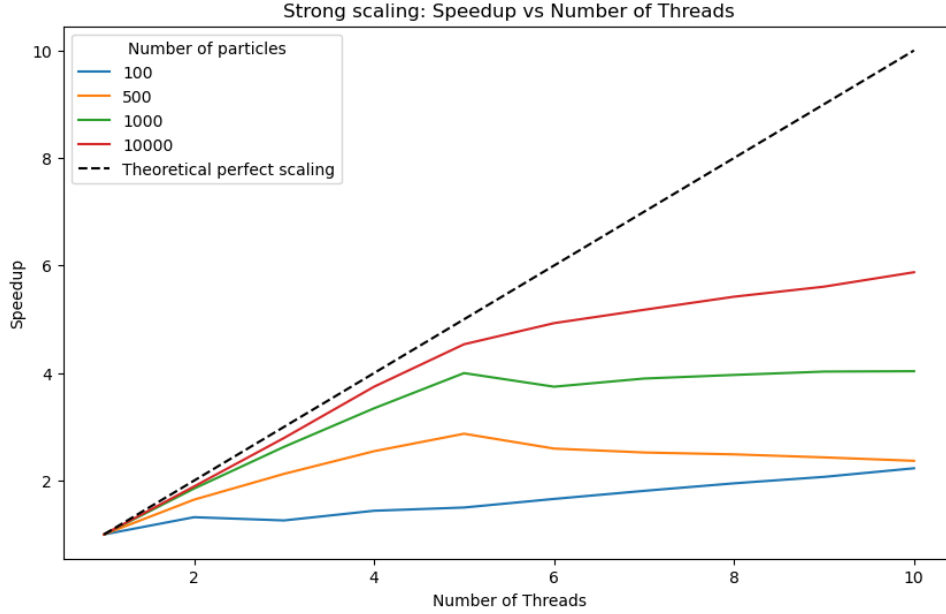


Figure 1: Strong scaling

For weak scaling we ran 500 iterations with 100 particles per thread for 1 to 10 threads. The datasets were picked with more and more particles for each step up in cores used. From the plot in figure 2 we see that we have relatively good weak scaling up until around 5 threads, where we see some diminishing returns again.

Hardware note: Both strong and weak scaling results were obtained on a 2023 Macbook Pro running an M3 Pro chip with 11 Cores.

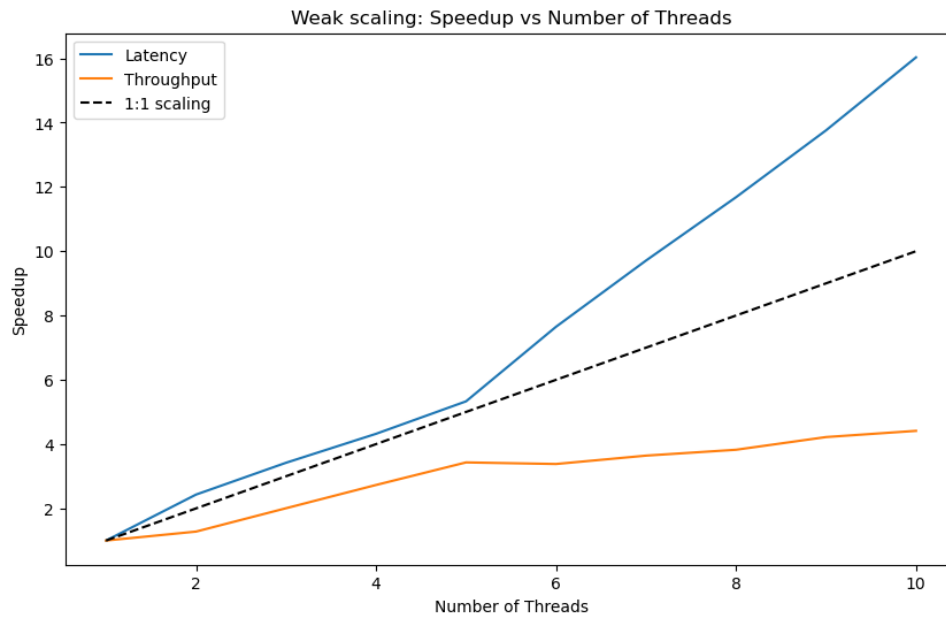


Figure 2: Weak scaling

- b) *Determine (in whatever way you find best) the performance overhead of updating the warning list.*

We created a copy of `nbody.c` where we had just removed the code updating the warnings in the `nbody()` function. We ran simulations of 1000 particles for 1000 iterations. There was no speedup in the updated version of the code, so the updating of the warnings themselves did not generate noticeable overhead. We ran a similar test for 10.000 particles with the same result. We note that this is an unexpected result and could suggest an issue.

- c) Are there workloads for which updating the warning list can have a ruinous impact on parallel scalability, and why?

Yes, by using the `#pragma omp critical` combined with the frequent memory reallocations, can disturb parallel scalability for workloads with high density.

- d) Do you have any memory leaks, and how can you know?

The program doesn't have any memory leaks. We know this by running the program with `valgrind` in the terminal. By analyzing the output we observe that in leak summary there is 0 definitely lost bytes. Another interesting thing from the output is that there is still 104 bytes reachable, which could mean that the program didn't free them before ending. This isn't meaning that there is an error because the memory is still relationed with valid pointers.

2. The following questions pertain to `nbody-bh.c`.

- a) Measure and show the strong scaling of `nbody-bh` as you vary the number of threads. Explain how you chose the datasets.

In strong scaling we are keeping the problem size constant and increasing core count to see how parallelization can improve performance. We expect to see diminishing returns in line with Almdal's law.

Surprisingly we don't see this at all, for our large dataset we get a linear scaling more or less, even for the 1 million point dataset. Potentially we could need more cores to see diminishing returns, I run 8 cores on my 4 multi-threaded cores. Hence 8 2.8GHZ intel i7 threads. We choose first to explore similar datasets to the naive set but with one larger million point set to see what would happen for very large datasets. However since these were run on different hardware this diagram is not comparable with Fig3

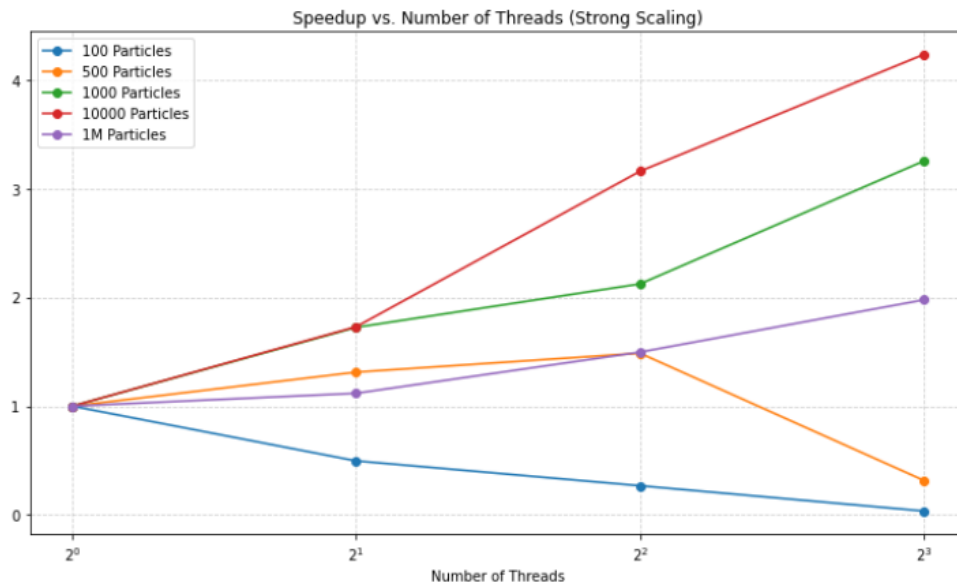


Figure 3: Strong Scaling Results for the N-Body Simulation

- b) Find a workload (changing either the number of particles or θ or both) where `nbody` is faster than `nbody-bh`. Explain why.

Barnes-Hut has the advantage that once the octree is built it is very fast to traverse, while there is a cost to building that tree. For very few particles it could be more efficient to just go the naive way and not pay that cost.

Barnes-Hut uses a parameter theta to determine to what extent it uses an approximation vs looking at the parent node and simulating that instead of all the children separately. A very small theta would essentially mean every leaf node has to be separately calculated just as in the naive simulation.

The combination therefore that could make the native version faster than the Barnes-Hut would be one with a very small theta and a very small number of particles. One example seen in fig 6 is for theta 0.1 and 10 particles, Where the native significantly outperforms. All simulations in the graph is run with very small theta and few particles but for larger theta and particles we would see Barnes-Hut outperform.

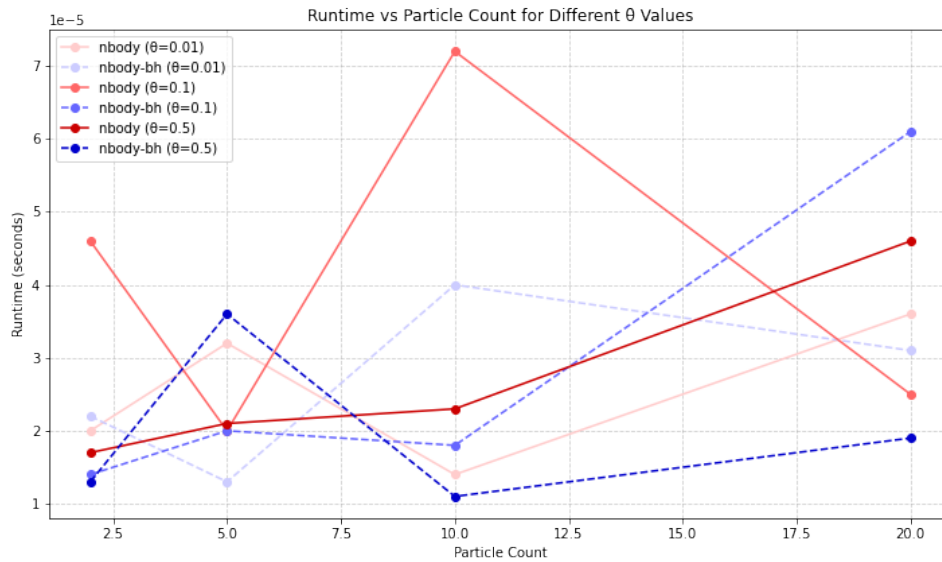


Figure 4: Strong Scaling Results for the N-Body Simulation

c) Do you have any memory leaks, and how can you know?

We check for this using valgrind like before and analyze the output, where we get the same. 0 definitely lost bytes in 0 blocks, and 104 bytes in 2 blocks that are still reachable. With this we conclude that there are no memory leaks.

Output from running benchmark.sh

The following results were obtained by running `benchmark.sh` on a M3 Pro Macbook Pro with 11 cores.

Measuring Strong Scaling

Speedups for nbody

```
1: 1.3025607348
2: 2.3268513626
4: 3.8787184169
8: 6.7542340391
```

Speedups for nbody-bh

```
1: .9877098650
2: 1.6135451025
4: 2.7792873463
8: 3.6726783091
```

Measuring Weak Scaling

Speedups for nbody

```
1: .9988305790
2: 1.8118361030
4: 3.2604482457
8: 5.1473073697
```

Speedups of nbody-bh vs nbody

```
500: .5277246653
1000: .6147704590
5000: 2.0319957574
10000: 3.5298413695
```